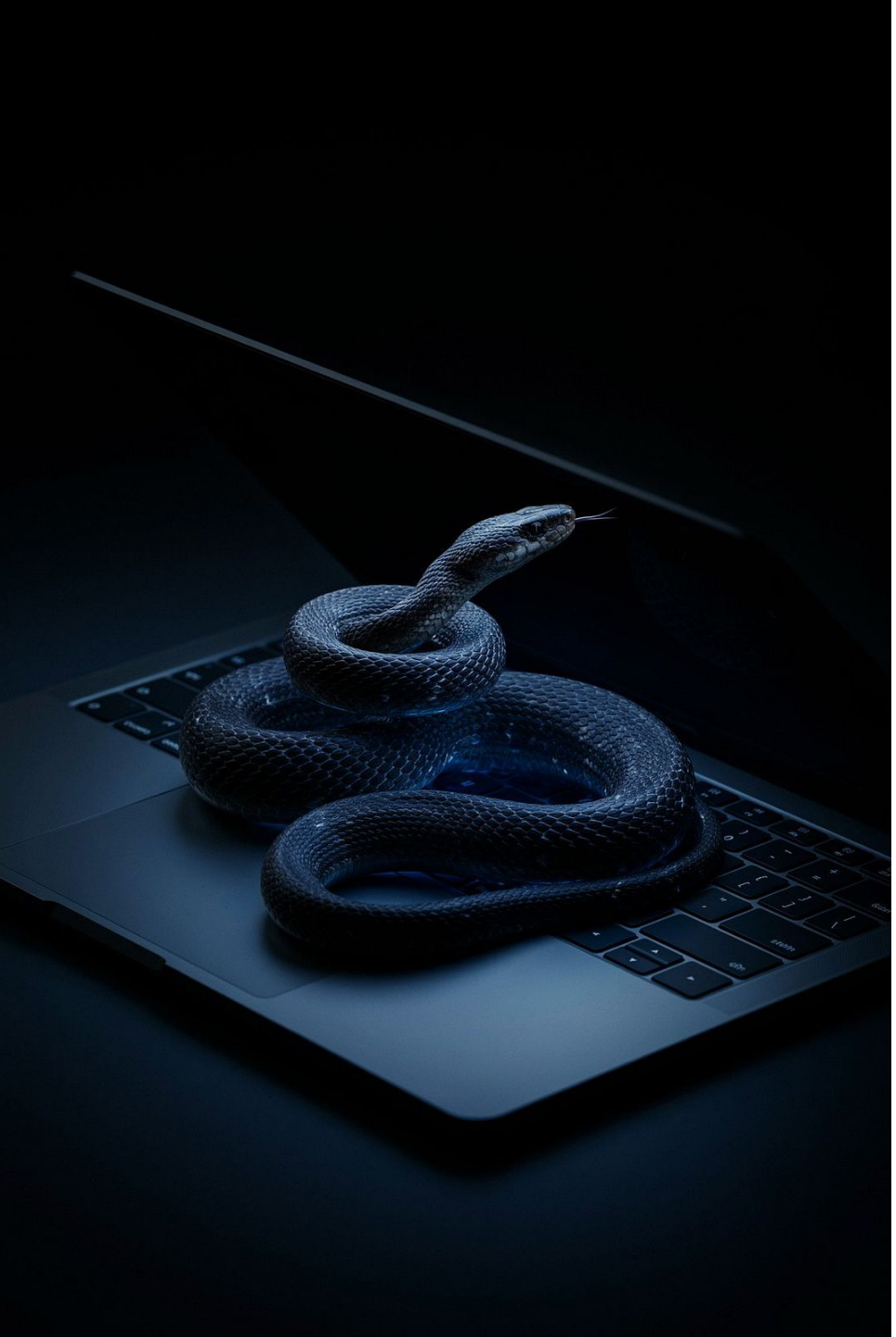


S4.1 · Python: Entorno y Lenguaje

Bienvenido a la sesión 4.1 del curso. Hoy sentamos las bases de Python, el lenguaje favorito del mundo de la Inteligencia Artificial. Aprenderemos a configurar nuestro entorno de trabajo, dominar la sintaxis esencial y escribir nuestros primeros programas.

NIVEL INICIAL · INTERMEDIO

PYTHON PARA IA

¿Por qué Python en IA?

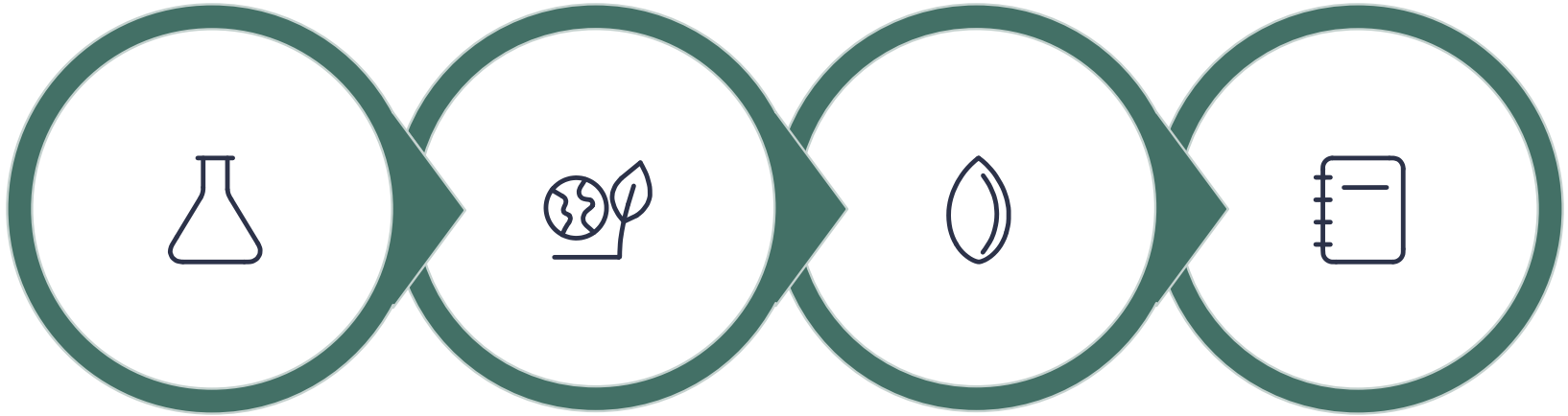
Python se ha convertido en el lenguaje **de facto** de la Inteligencia Artificial. No es casualidad: su diseño prioriza la legibilidad y la productividad, lo que permite a investigadores y desarrolladores centrarse en los problemas, no en la sintaxis.



ENTORNO DE TRABAJO

Configurando tu entorno Python

Un entorno bien configurado es la base de cualquier proyecto. Antes de escribir una sola línea de código IA, necesitamos instalar y entender las herramientas que usaremos a diario.



Instalar Python

Crear venv

Instalar paquetes

Abrir Jupyter/VS Code

Las herramientas del ecosistema

🐍 Python

El intérprete del lenguaje. Descárgalo desde **python.org** (versión 3.10 o superior). Incluye la librería estándar y el gestor de paquetes `pip`.

📦 Entornos virtuales

Un entorno virtual (`venv`) aísla las dependencias de cada proyecto. Así evitas conflictos entre versiones de paquetes. Créalo con `python -m venv .venv` y actívalo con `source .venv/bin/activate`.

📬 pip

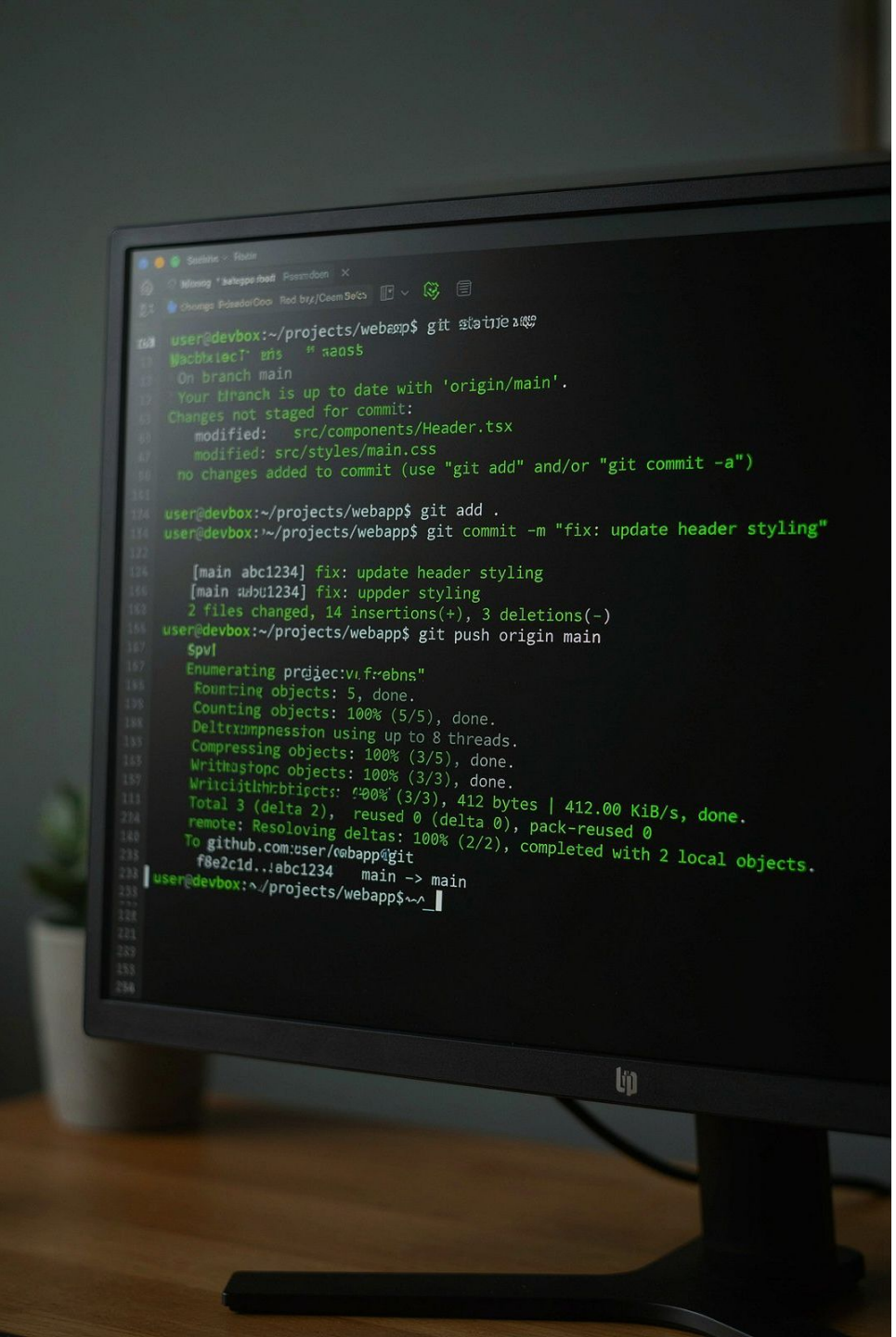
El gestor de paquetes de Python. Instala bibliotecas con `pip install nombre_paquete`. Guarda las dependencias del proyecto con `pip freeze > requirements.txt`.

📓 Jupyter Notebook

Entorno interactivo que combina código, texto y gráficos en un mismo documento. Perfecto para exploración de datos y experimentos de IA. Instálalo con `pip install notebook`.

🖥️ VS Code

Editor de código ligero y muy potente. Con la extensión de Python ofrece autocompletado, depuración y soporte para Jupyter integrado.



Git básico: tu historial de cambios

Git es el sistema de control de versiones más utilizado del mundo. Te permite guardar «fotografías» de tu código a lo largo del tiempo, colaborar en equipo y recuperar versiones anteriores si algo sale mal.

1

git init

Inicializa un repositorio en tu carpeta de proyecto.

2

git add

Marca los ficheros modificados que quieres guardar.

3

git commit

Guarda la «fotografía» con un mensaje descriptivo.

4

git push

Sube los cambios al repositorio remoto (p. ej. GitHub).

VARIABLES Y TIPOS

Variables: cajas para guardar datos

Una variable es un nombre que apunta a un valor almacenado en memoria. En Python **no necesitas declarar el tipo** explícitamente; el intérprete lo deduce solo. Esto se llama *tipado dinámico*.

```
nombre = "Ada Lovelace"   # str  → cadena de texto
edad   = 28               # int  → número entero
altura = 1.72             # float → decimal
activo = True             # bool → verdadero/falso
nada   = None             # NoneType → ausencia de valor
```

ⓘ Las variables en Python distinguen mayúsculas de minúsculas: `edad` y `Edad` son variables diferentes.

Tipos de datos fundamentales

int

Números enteros sin decimales.

`x = 42`

`año = 2025`

float

Números con parte decimal.

`pi = 3.1416`

`tasa = 0.05`

str

Cadenas de texto entre comillas.

`"Hola"`

`'Python'`

bool

Verdadero o falso.

`True`

`False`

NoneType

Representa la ausencia de valor.

`resultado = None`

Puedes verificar el tipo de cualquier variable con la función incorporada `type(variable)`.

OPERADORES

Operadores en Python

Los operadores son los símbolos que permiten realizar cálculos, comparaciones y operaciones lógicas. Conocerlos bien es imprescindible para escribir cualquier algoritmo.

Operadores aritméticos

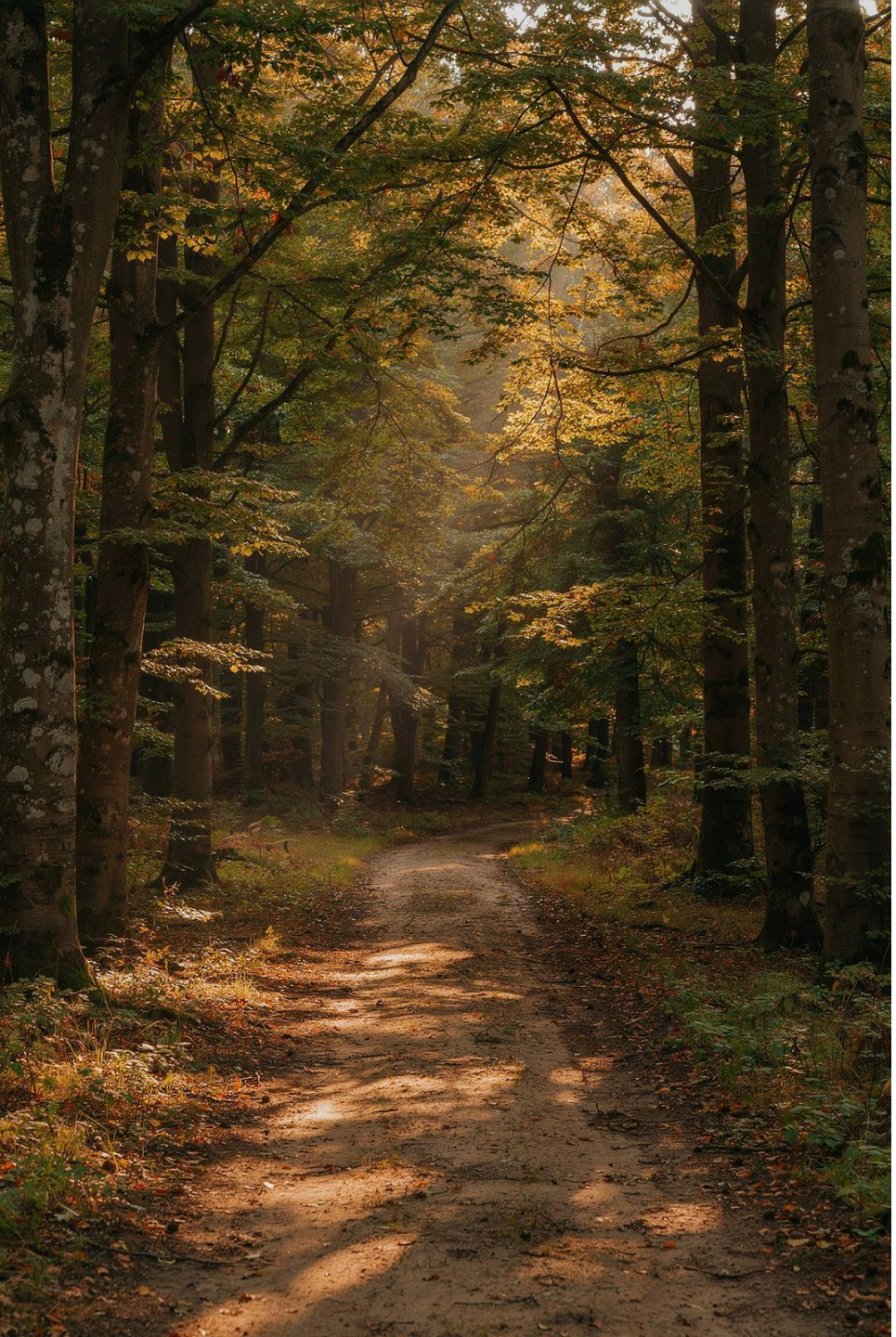
```
+   suma
-   resta
*   multiplicación
/   división (float)
//  división entera
%   módulo (resto)
**  potencia
```

Operadores de comparación

```
==  igual
!=  distinto
>   mayor que
<   menor que
>=  mayor o igual
<=  menor o igual
```

Operadores lógicos

```
and y lógico
or o lógico
not negación
```



CONTROL DE FLUJO

Condicionales: tomando decisiones

Los condicionales permiten que tu programa ejecute diferentes bloques de código según se cumplan o no ciertas condiciones. En Python se escriben con `if`, `elif` y `else`, y la **indentación** (sangría) define los bloques — no se usan llaves como en Java.

Bucles: repitiendo acciones

Los bucles ejecutan un bloque de código múltiples veces. Python ofrece dos tipos principales:

Bucle `for`

Itera sobre una secuencia (lista, rango, cadena...). Sabes de antemano cuántas veces se ejecutará.

```
for i in range(5):
    print(i)  # 0 1 2 3 4

frutas = ["manzana","pera","uva"]
for fruta in frutas:
    print(fruta)
```

Bucle `while`

Repite mientras una condición sea verdadera. Útil cuando no sabes cuántas iteraciones necesitarás.

```
contador = 0
while contador < 3:
    print(contador)
    contador += 1
# Resultado: 0 1 2
```

⚠ Un bucle `while` sin condición de salida bien definida puede ejecutarse infinitamente y bloquear tu programa.

ENTRADA Y SALIDA

Comunicándose con el usuario

Todo programa útil necesita recibir datos del exterior y mostrar resultados. Python lo hace de manera muy sencilla con dos funciones integradas: `print()` e `input()`.

Salida: `print()`

```
# Texto simple
print("Hola, mundo")

# Variables
nombre = "Lucía"
print("Hola,", nombre)

# f-strings (recomendado)
edad = 21
print(f"{nombre} tiene {edad} años")
```

Entrada: `input()`

```
# Siempre devuelve str
nombre = input("¿Cómo te llamas? ")

# Convierte a número si lo necesitas
edad = int(input("¿Cuántos años tienes? "))

print(f"¡Hola {nombre}!")
print(f"En 10 años tendrás {edad+10}")
```

Importante: `input()` siempre devuelve texto. Usa `int()` o `float()` para convertirlo.

S04 · EN EL REPOSITORIO DEL CURSO

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO **S04_01_entorno_sintaxis** — 45 min
Entorno, variables, tipos, if, for y while
sprints/S04/notebooks/S04_01_entorno_sintaxis.ipynb

CUADERNO **S04_02_funciones_modulos** — 45 min
Funciones, *args, lambda y módulos
sprints/S04/notebooks/S04_02_funciones_modulos.ipynb

CUADERNO **S04_03_estructuras_datos** — 45 min
Listas, tuplas, diccionarios y conjuntos
sprints/S04/notebooks/S04_03_estructuras_datos.ipynb

CUADERNO **S04_04_ficheros_excepciones** — 35 min
CSV, JSON y try/except (prepara leer_reservas)
sprints/S04/notebooks/S04_04_ficheros_excepciones.ipynb